

TurnosRed — Informe técnico

Evidencia de depuración, pruebas de API y eventos en tiempo real · Actividad de Integraciones Web

Repositorio: github.com/Luismcbo/turnos-red · **Stack:** Node.js + TypeScript + Express + Socket.IO · **Autor:** Luis Valenzuela

Este informe documenta tres piezas de evidencia sobre el backend TurnosRed: una sesión de depuración en VS Code sobre el endpoint **POST /turnos**, las pruebas de los 5 endpoints REST en Postman (200/201/400/404), y la recepción en vivo de eventos por Socket.IO al disparar operaciones desde Postman/curl, sin polling.

1. Sesión de depuración (VS Code)

Se colocó un breakpoint en `src/controllers/turno.controller.ts:59`, dentro de `crearTurno`, justo antes de persistir el turno normalizado. Al enviar un **POST /turnos**, la ejecución se detiene ahí y el panel de Variables muestra el `bodyValido` y el Call Stack confirma "Paused On Breakpoint". Con **Step Over (F10)** se avanza a la línea siguiente, donde ya existe `turnoCreado` antes de responder con 201.

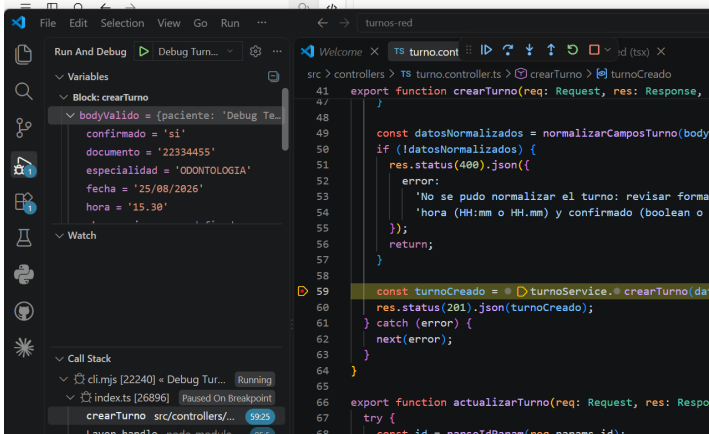


Fig. 1 — Breakpoint alcanzado en `crearTurno` (línea 59). Variables: `bodyValido` expandido.

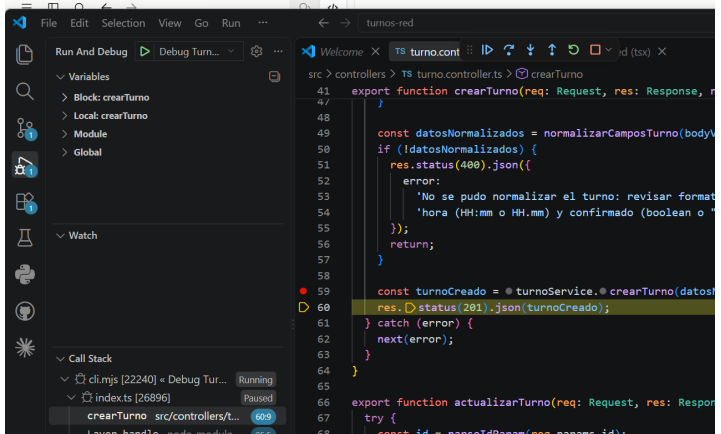


Fig. 2 — **Step Over (F10)**: avance a la línea 60 con `turnoCreado` ya disponible.

2. Pruebas de los 5 endpoints en Postman

Colección **TurnosRed API** contra `http://localhost:3000`. A continuación, **GET /turnos** (200, lista completa), **GET /turnos/:id** (200, un turno) y **GET /turnos/:id** con id inexistente (404).

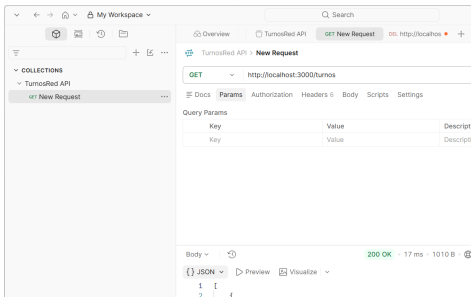


Fig. 3 — GET /turnos → 200 OK

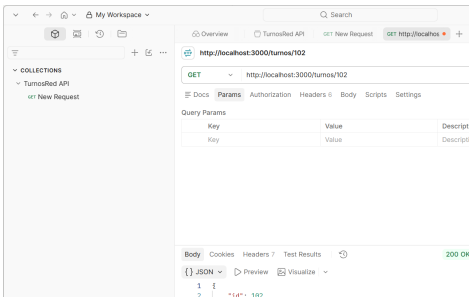


Fig. 4 — GET /turnos/102 → 200 OK

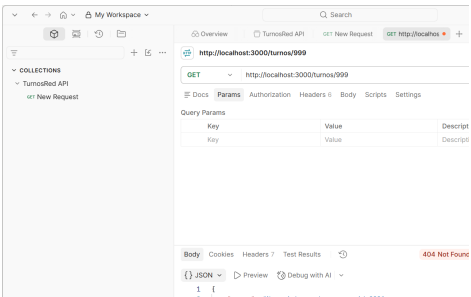


Fig. 5 — GET /turnos/999 → 404

POST /turnos: creación exitosa (201, con normalización de fecha/hora/confirmado) y body inválido (400, falta un campo requerido).

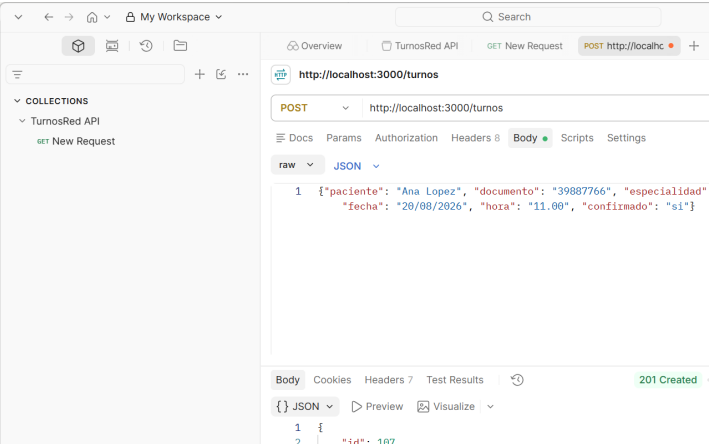


Fig. 6 — POST /turnos → 201 Created

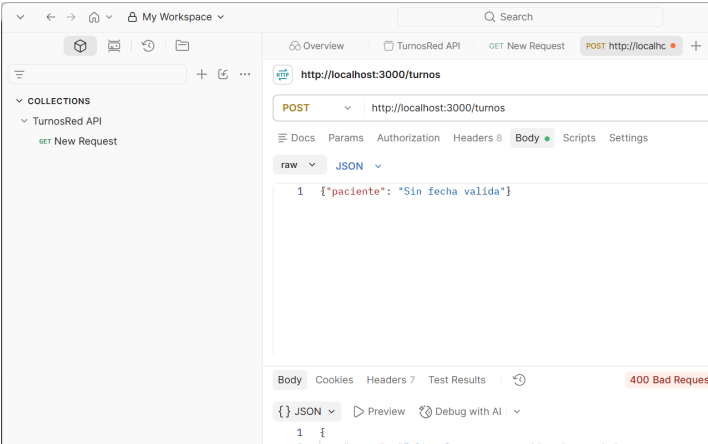


Fig. 7 — POST /turnos (body incompleto) → 400 Bad Request

2. Pruebas de los 5 endpoints en Postman (cont.)

PUT /turnos/:id: actualización exitosa (200) e id inexistente (404).

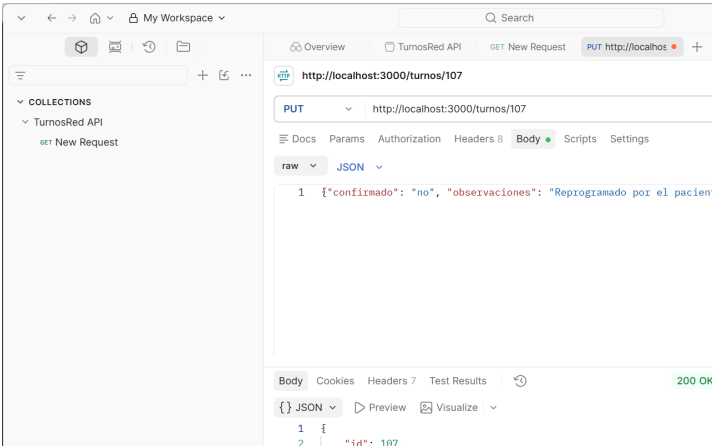


Fig. 8 — PUT /turnos/107 → 200 OK

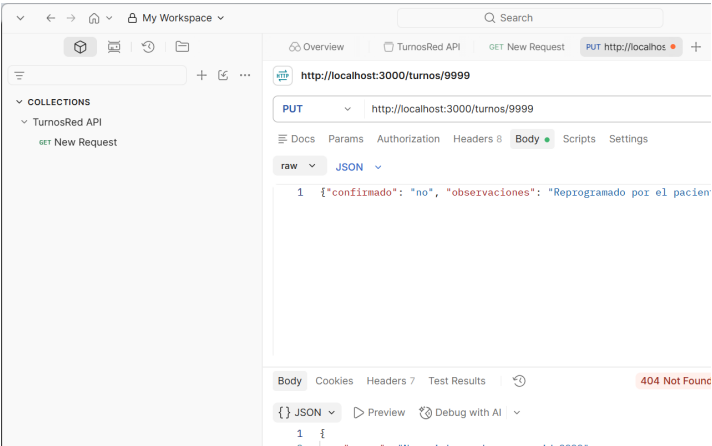


Fig. 9 — PUT /turnos/9999 → 404 Not Found

DELETE /turnos/:id: eliminación exitosa (200, devuelve el turno eliminado) y segunda eliminación del mismo id (404).

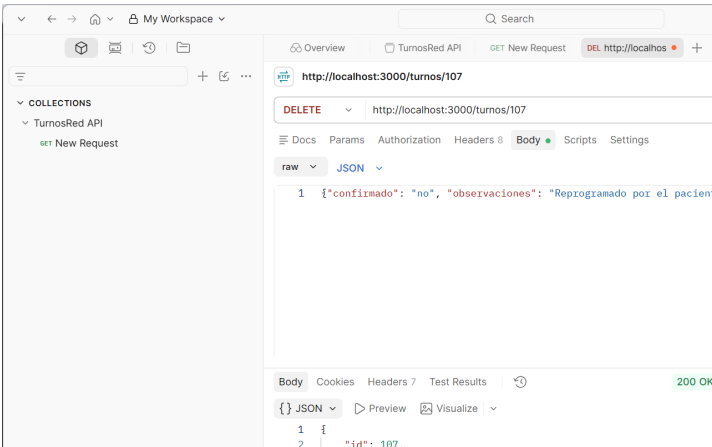


Fig. 10 — DELETE /turnos/107 → 200 OK

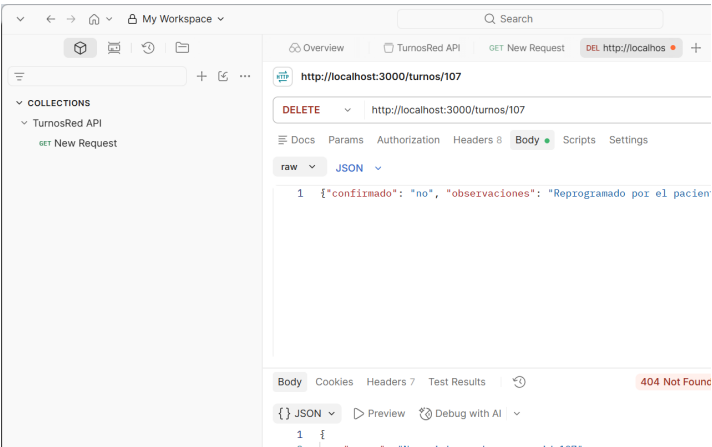


Fig. 11 — DELETE /turnos/107 (repetido) → 404 Not Found

3. Eventos en tiempo real (Socket.IO)

Cada operación exitosa del servicio de turnos emite un evento por un EventEmitter interno (turno:creado, turno:actualizado, turno:eliminado). Socket.IO está suscripto a ese bus y retransmite turno:nuevo, turno:actualizado y turno:eliminado a todos los clientes conectados. El cliente de prueba (public/index.html) abre el WebSocket una sola vez y muestra cada evento apenas ocurre en el servidor — **sin polling**. La captura muestra los tres eventos recibidos en vivo tras un POST, un PUT y un DELETE consecutivos contra /turnos.

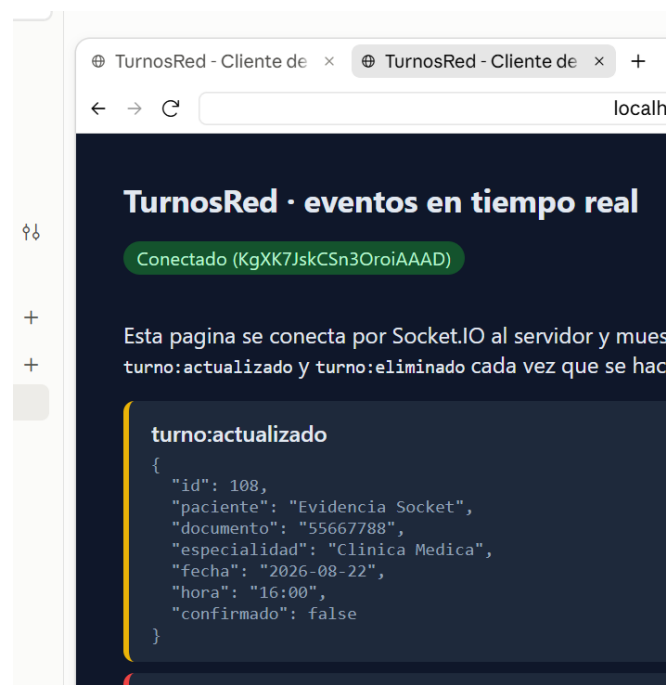


Fig. 12 — Cliente Socket.IO recibiendo turno:actualizado, turno:eliminado y turno:nuevo en vivo, sin recargar la pagina.

4. Resumen de endpoints verificados

Metodo	Ruta	Casos probados	Resultado
GET	/turnos	Listado completo	200 OK
GET	/turnos/:id	Existente / inexistente	200 OK / 404 Not Found
POST	/turnos	Body valido / incompleto	201 Created / 400 Bad Request
PUT	/turnos/:id	Existente / inexistente	200 OK / 404 Not Found
DELETE	/turnos/:id	Existente / ya eliminado	200 OK / 404 Not Found

Los errores no controlados responden **500** con { "error": "Error interno del servidor" } desde el middleware centralizado en src/middlewares/errorHandler.ts.

Conclusión: los cinco endpoints REST de TurnosRed responden con los códigos de estado esperados en sus casos de éxito y de error, la normalización de datos heterogéneos funciona correctamente (verificado en debugging), y la notificación en tiempo real por Socket.IO llega a los clientes conectados inmediatamente después de cada operación exitosa, sin necesidad de polling.